

Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa – IDP

Disciplina: Redes de Computadores e Internet

Professora: Lorena Borges

Plataforma de Monitoramento para DevOps

Monitoramento de Web Server, Banco de Dados, DNS, SMTP e Eventos de
Segurança

Integrantes:

André Barros Soares

Pietro Branco Rossi

Brasília – DF

2026

Sumário

1	Introdução	3
2	Objetivos	4
2.1	Objetivo Geral	4
2.2	Objetivos Específicos	4
3	Requisitos do Trabalho	4
4	Arquitetura da Solução	5
4.1	Componentes Principais	5
4.2	Fluxo de Dados	6
5	Tecnologias Utilizadas	7
5.1	C++ e Crow	7
5.2	Docker Compose	7
5.3	Prometheus	7
5.4	Grafana	7
5.5	Alertmanager e MailHog	7
5.6	PostgreSQL e Postgres Exporter	7
5.7	Blackbox Exporter	8
6	Implementação do Backend	8
7	Front-end da Plataforma	8
7.1	Dashboard Principal	9
7.2	Laboratório de Ataques	9
8	Banco de Dados	9
9	Métricas Monitoradas	9
9.1	Web Server	9
9.2	Banco de Dados	10
9.3	DNS	10
9.4	SMTP	10

9.5	Segurança	10
10	Regras de Alerta	11
11	Notificações por E-mail	11
12	Execução do Projeto	12
13	Testes e Validação	12
13.1	Teste de Navegação	12
13.2	Teste de Carga HTTP	13
13.3	Teste de Falhas de Autenticação	13
13.4	Teste de Vulnerabilidade Conhecida	13
13.5	Teste de Alteração de Configuração	13
14	Runbook de Operação	13
14.1	Subir a Plataforma	13
14.2	Verificar Containers	14
14.3	Consultar Logs	14
14.4	Parar a Plataforma	14
15	Playbooks de Incidentes	14
15.1	CPU Alta	14
15.2	Erros HTTP	14
15.3	Banco Indisponível	14
15.4	Falhas de Autenticação	14
15.5	Vulnerabilidade Conhecida	15
16	Resultados	15
17	Limitações	15
18	Trabalhos Futuros	15
19	Conclusão	16

Resumo

Este relatório apresenta o desenvolvimento de uma plataforma de monitoramento para DevOps, construída para acompanhar serviços de rede e componentes de infraestrutura em ambiente local. A solução monitora um servidor web desenvolvido em C++, um banco de dados PostgreSQL, serviços DNS e SMTP, além de eventos relacionados à segurança, como picos de tráfego, falhas de autenticação, alterações de configuração e vulnerabilidades conhecidas simuladas. A arquitetura utiliza Docker Compose, Prometheus, Grafana, Alertmanager, MailHog, Postgres Exporter e Blackbox Exporter. O resultado é um ambiente integrado para coleta de métricas, visualização em dashboards, emissão de alertas por e-mail e execução de testes controlados.

Palavras-chave: redes de computadores; monitoramento; DevOps; Prometheus; Grafana; segurança.

1 Introdução

Ambientes computacionais modernos dependem de diversos serviços operando de forma integrada. Servidores web, bancos de dados, serviços de resolução de nomes, canais de e-mail e mecanismos de autenticação precisam estar disponíveis e funcionando com desempenho adequado. Quando um desses componentes apresenta falha, lentidão ou comportamento anormal, a equipe responsável precisa identificar rapidamente a causa e responder ao incidente.

Nesse contexto, plataformas de monitoramento são fundamentais. Elas permitem acompanhar métricas de disponibilidade, latência, volume de acessos, taxa de erros, conexões ativas e eventos de segurança. Além disso, dashboards e alertas automatizados facilitam a observação contínua do ambiente e reduzem o tempo necessário para detectar e tratar problemas.

O projeto desenvolvido consiste em uma plataforma de monitoramento para DevOps, com foco em serviços de rede. A solução foi estruturada para atender aos requisitos propostos na disciplina de Redes de Computadores e Internet, contemplando front-end responsivo, dashboards, backend de métricas, banco de dados, regras de alerta, notificações por e-mail e documentação operacional.

2 Objetivos

2.1 Objetivo Geral

Projetar e implementar uma plataforma de monitoramento capaz de acompanhar a saúde de serviços de rede e infraestrutura, apresentando métricas em dashboards e notificando situações de degradação ou indisponibilidade.

2.2 Objetivos Específicos

- Implementar um front-end responsivo com visão consolidada dos serviços monitorados.
- Desenvolver um backend em C++ para simular um serviço web e expor métricas em formato compatível com Prometheus.
- Adicionar um banco de dados PostgreSQL ao ambiente e monitorar suas métricas.
- Monitorar disponibilidade e tempo de resposta de serviços HTTP, DNS e SMTP.
- Configurar dashboards no Grafana para visualização das métricas.
- Criar regras de alerta com níveis de atenção e criticidade.
- Enviar notificações de alerta por e-mail em ambiente local.
- Implementar cenários de teste relacionados a segurança e comportamento anormal.
- Documentar arquitetura, execução, testes e resposta a incidentes.

3 Requisitos do Trabalho

O enunciado do trabalho solicita uma plataforma de monitoramento de serviços de rede, com dashboards, coleta e armazenamento de métricas, alertas e documentação. A Tabela 1 apresenta a relação entre os requisitos e a implementação realizada.

Tabela 1: Atendimento aos requisitos do trabalho.

Requisito	Implementação realizada
Front-end responsivo	Dashboard próprio servido pelo backend C++ em <code>http://localhost:8080</code> .

Requisito	Implementação realizada
Dashboards por serviço	Dashboard no Grafana com métricas de web server, banco de dados, DNS, SMTP e segurança.
Visão consolidada	Tela principal com resumo dos indicadores essenciais e links para Prometheus, Grafana, MailHog e laboratório de testes.
Backend de ingestão e armazenamento	Backend C++ expõe métricas; Prometheus coleta e armazena séries temporais; PostgreSQL representa a camada de armazenamento operacional.
Regras de alerta	Arquivo <code>condicoes_alerta.yml</code> com alertas para CPU, HTTP, banco, DNS, SMTP e eventos de segurança.
Notificação por e-mail	Alertmanager configurado para enviar alertas ao MailHog, que simula uma caixa de entrada local.
Documentação	Arquivos de arquitetura, runbook, playbooks, roteiro de vídeo e relatório técnico.
Testes de navegação e uso	Laboratório de ataques e roteiro de testes para demonstração dos alertas e dashboards.
Monitoramentos extras	Deteção de picos de tráfego, falhas de autenticação, alterações de configuração e vulnerabilidades conhecidas simuladas.

4 Arquitetura da Solução

A plataforma foi estruturada com serviços em contêineres, utilizando Docker Compose para facilitar a execução e a reprodução do ambiente. Essa abordagem permite subir todos os componentes necessários com um único comando, evitando configurações manuais extensas.

4.1 Componentes Principais

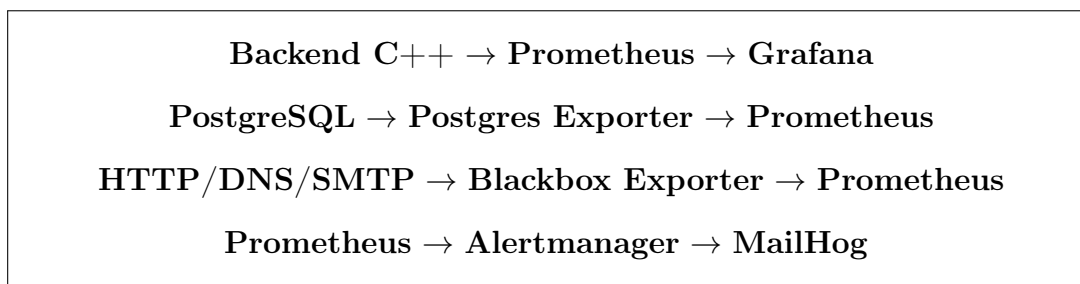
Tabela 2: Componentes da arquitetura.

Componente	Responsabilidade
Backend C++	Servir o front-end, simular a aplicação web, expor métricas e fornecer rotas de simulação de incidentes.

Componente	Responsabilidade
PostgreSQL	Banco de dados da plataforma, inicializado com tabelas para métricas e incidentes.
Postgres Exporter	Exportar métricas internas do PostgreSQL para o Prometheus.
Prometheus	Coletar métricas, armazenar séries temporais e avaliar regras de alerta.
Grafana	Exibir dashboards operacionais com base nos dados coletados pelo Prometheus.
Alertmanager	Receber alertas do Prometheus e encaminhar notificações.
MailHog	Simular um servidor SMTP local para captura dos e-mails de alerta.
Blackbox Exporter	Executar probes de disponibilidade e latência para HTTP, DNS e SMTP.

4.2 Fluxo de Dados

O backend C++ expõe métricas no endpoint `/metrics`. O Prometheus coleta essas métricas periodicamente, juntamente com dados do Postgres Exporter e do Blackbox Exporter. O Grafana consulta o Prometheus para renderizar os dashboards. Quando uma condição de alerta é atendida, o Prometheus encaminha o alerta ao Alertmanager. O Alertmanager, por sua vez, envia a notificação para o MailHog, permitindo visualizar os e-mails em ambiente local.



5 Tecnologias Utilizadas

5.1 C++ e Crow

O backend foi implementado em C++ com a biblioteca Crow. A aplicação simula um servidor web real, processa requisições, registra contadores e expõe métricas em texto no formato utilizado pelo Prometheus.

5.2 Docker Compose

O Docker Compose foi utilizado para orquestrar todos os serviços do ambiente. A plataforma inclui contêineres para backend, PostgreSQL, Postgres Exporter, Prometheus, Blackbox Exporter, Alertmanager, MailHog e Grafana.

5.3 Prometheus

O Prometheus atua como mecanismo central de coleta e armazenamento de séries temporais. Ele executa scraping periódico dos endpoints configurados e avalia as regras de alerta.

5.4 Grafana

O Grafana é responsável pela visualização das métricas. O projeto utiliza provisionamento automático de datasource e dashboard, permitindo que os painéis estejam disponíveis assim que o ambiente é iniciado.

5.5 Alertmanager e MailHog

O Alertmanager organiza e encaminha alertas. O MailHog foi utilizado como servidor SMTP local para capturar as notificações, possibilitando demonstrar o envio de e-mails sem depender de uma conta real.

5.6 PostgreSQL e Postgres Exporter

O PostgreSQL representa o banco de dados da plataforma. O Postgres Exporter coleta métricas do banco e as disponibiliza para o Prometheus.

5.7 Blackbox Exporter

O Blackbox Exporter executa testes externos de disponibilidade e tempo de resposta. No projeto, ele é utilizado para monitorar HTTP, DNS e SMTP.

6 Implementação do Backend

O backend C++ disponibiliza tanto a interface visual quanto os endpoints de métricas e simulação. As principais rotas implementadas são:

- `/`: dashboard principal da plataforma.
- `/ataques`: laboratório de ataques e simulações controladas.
- `/api/trabalho`: rota que simula a aplicação web monitorada.
- `/api/simular/login-falho`: simula falhas de autenticação.
- `/api/simular/configuracao`: simula alteração de configuração.
- `/api/simular/vulnerabilidade`: ativa uma vulnerabilidade conhecida simulada.
- `/api/simular/vulnerabilidade/resolver`: resolve a vulnerabilidade simulada.
- `/metrics`: expõe as métricas consumidas pelo Prometheus.

A rota `/api/trabalho` simula o comportamento de um serviço web. Ela incrementa contadores de requisições, registra conexões ativas, calcula latência e gera erros HTTP de forma controlada. Com isso, é possível observar variações nas métricas durante os testes.

7 Front-end da Plataforma

O projeto possui duas telas principais. A primeira é o painel de monitoramento em `http://localhost:8080`. Essa tela apresenta os indicadores principais do ambiente, incluindo disponibilidade, CPU, requisições, erros HTTP, latência, conexões ativas, falhas de login e vulnerabilidades.

A segunda tela é o laboratório de ataques, disponível em `http://localhost:8080/ataques`. Ela foi criada para facilitar a apresentação do trabalho e permitir a geração de eventos diretamente pelo navegador.

7.1 Dashboard Principal

O dashboard principal apresenta uma visão consolidada dos serviços monitorados. Ele também contém atalhos para Prometheus, Grafana, MailHog e laboratório de ataques. A interface foi construída de forma responsiva, permitindo uso em diferentes tamanhos de tela.

7.2 Laboratório de Ataques

O laboratório de ataques permite executar testes controlados contra a própria aplicação. Os cenários disponíveis são:

- Pico HTTP ou DDoS local.
- Brute force de login.
- Alteração de configuração simulada.
- Vulnerabilidade conhecida simulada.

Essa tela também apresenta métricas ao vivo, permitindo observar o efeito das simulações sobre os contadores expostos pelo backend.

8 Banco de Dados

O banco de dados utilizado é o PostgreSQL. Ele é iniciado automaticamente pelo Docker Compose e recebe um script de inicialização com tabelas para métricas e incidentes. A inclusão do banco atende ao requisito de armazenamento da plataforma e permite monitorar um serviço adicional.

As principais informações monitoradas no banco incluem disponibilidade, número de conexões abertas, transações, rollbacks e tamanho do banco.

9 Métricas Monitoradas

9.1 Web Server

O servidor web C++ expõe as seguintes métricas:

- `servidor_web_up`: indica se o servidor está disponível.

- `total_requisicoes_http`: total acumulado de requisições.
- `total_erros_http`: total acumulado de erros HTTP.
- `ultima_latencia_ms`: latência da última resposta em milissegundos.
- `conexoes_ativas`: quantidade de conexões ativas no momento.

9.2 Banco de Dados

O monitoramento do PostgreSQL é feito pelo Postgres Exporter. As principais métricas utilizadas são:

- `pg_up`: disponibilidade do banco.
- `pg_stat_activity_count`: número de conexões abertas.
- `pg_stat_database_xact_commit`: transações confirmadas.
- `pg_stat_database_xact_rollback`: transações revertidas.
- `pg_database_size_bytes`: tamanho do banco.

9.3 DNS

O DNS é monitorado por meio do Blackbox Exporter. O probe verifica se uma consulta DNS pode ser resolvida corretamente e se o serviço responde dentro do tempo esperado.

9.4 SMTP

O SMTP é monitorado por um probe TCP direcionado ao MailHog. Dessa forma, é possível verificar se o canal de envio de e-mails utilizado pelo Alertmanager está disponível.

9.5 Segurança

Os eventos de segurança monitorados são:

- Pico de tráfego, utilizado como indicador de possível ataque de negação de serviço.
- Falhas de autenticação, associadas a tentativas de brute force.
- Alterações de configuração, detectadas por meio de checksum.
- Vulnerabilidade conhecida simulada, usada para demonstrar um alerta crítico.

10 Regras de Alerta

As regras de alerta foram configuradas no arquivo `condicoes_alerta.yml`. Elas são avaliadas periodicamente pelo Prometheus. Quando uma condição é atendida durante o tempo configurado, o alerta é enviado ao Alertmanager.

Tabela 3: Principais alertas implementados.

Alerta	Severidade	Condição monitorada
Alerta_CPU_Amarelo	Warning	CPU acima de 70%.
Alerta_CPU_Vermelho	Critical	CPU acima de 90%.
Alta_Taxa_Erros_HTTP	Critical	Erros HTTP detectados na aplicação.
Banco_Indisponivel	Critical	PostgreSQL sem resposta pelo exporter.
Banco_Muitas_Conexoes	Warning	Número elevado de conexões abertas.
Banco_Alta_Taxa_Rollbacks	Warning	Rollbacks no banco de dados.
Servico_HTTP_Indisponivel	Critical	Probe HTTP falhando.
SMTP_Indisponivel	Critical	SMTP local sem resposta.
DNS_Indisponivel	Warning	Falha no teste DNS.
Possivel_DDoS_HTTP	Critical	Volume elevado de requisições.
Falhas_Autenticacao_Elevadas	Warning	Muitas falhas de login em curto período.
Alteracao_Configuracao	Warning	Mudança detectada em checksum de configuração.
Vulnerabilidade_Conhecida	Critical	Vulnerabilidade simulada ativa.

11 Notificações por E-mail

O envio de notificações é feito pelo Alertmanager. No ambiente de desenvolvimento, o destino configurado é o MailHog, que recebe as mensagens em uma interface web local. Essa escolha permite demonstrar o funcionamento dos alertas sem depender de serviços externos de e-mail.

O fluxo de notificação ocorre da seguinte forma:

1. O Prometheus avalia uma regra de alerta.
2. A condição permanece ativa pelo tempo configurado.
3. O alerta é enviado ao Alertmanager.
4. O Alertmanager encaminha o e-mail ao MailHog.
5. O e-mail pode ser visualizado em `http://localhost:8025`.

12 Execução do Projeto

Para executar a plataforma completa, utiliza-se o comando:

```
docker compose up -d --build
```

Após a execução, os serviços ficam disponíveis nos endereços apresentados na Tabela 4.

Tabela 4: Endereços locais da plataforma.

Serviço	URL	Finalidade
Front principal	http://localhost:8080	Visão consolidada.
Laboratório	http://localhost:8080/ataques	Testes e simulações.
Prometheus	http://localhost:9090	Métricas e regras.
Grafana	http://localhost:3000	Dashboards.
Alertmanager	http://localhost:9093	Gerência de alertas.
MailHog	http://localhost:8025	Caixa de e-mails local.
Postgres Exporter	http://localhost:9187	Métricas do banco.
Blackbox Exporter	http://localhost:9115	Probes de rede.

13 Testes e Validação

A validação da plataforma envolve navegação, consulta de métricas, geração de carga e acionamento de alertas. Os testes podem ser feitos pelo laboratório de ataques ou por linha de comando.

13.1 Teste de Navegação

1. Abrir o front principal em <http://localhost:8080>.
2. Verificar os cards de métricas.
3. Abrir o laboratório em <http://localhost:8080/ataques>.
4. Abrir o Grafana em <http://localhost:3000>.
5. Consultar os targets do Prometheus em <http://localhost:9090/targets>.
6. Abrir o MailHog em <http://localhost:8025>.

13.2 Teste de Carga HTTP

O teste de carga HTTP pode ser executado pelo laboratório de ataques ou pelo comando:

```
ab -n 20000 -c 100 http://localhost:8080/api/trabalho
```

Esse teste aumenta o volume de requisições e permite observar RPS, latência, conexões ativas e erros HTTP.

13.3 Teste de Falhas de Autenticação

```
for i in {1..6}; do curl -i http://localhost:8080/api/simular/login-falho; done
```

Esse teste incrementa a métrica de falhas de login e permite acionar o alerta de autenticação.

13.4 Teste de Vulnerabilidade Conhecida

```
curl http://localhost:8080/api/simular/vulnerabilidade  
curl http://localhost:8080/api/simular/vulnerabilidade/resolver
```

O primeiro comando ativa a vulnerabilidade simulada, enquanto o segundo remove a condição crítica.

13.5 Teste de Alteração de Configuração

```
curl http://localhost:8080/api/simular/configuracao
```

Esse teste altera uma métrica de checksum simulada e permite demonstrar o alerta de alteração de configuração.

14 Runbook de Operação

O runbook define ações básicas para operar o ambiente.

14.1 Subir a Plataforma

```
docker compose up -d --build
```

14.2 Verificar Containers

```
docker compose ps
```

14.3 Consultar Logs

```
docker compose logs backend  
docker compose logs prometheus  
docker compose logs grafana
```

14.4 Parar a Plataforma

```
docker compose down
```

15 Playbooks de Incidentes

15.1 CPU Alta

Quando houver alerta de CPU alta, deve-se verificar se há teste de carga em andamento, consultar os gráficos do Grafana e analisar os logs do backend.

15.2 Erros HTTP

Em caso de aumento de erros HTTP, deve-se verificar a rota `/api/trabalho`, observar a taxa de erro no Grafana e confirmar se há simulação ativa no laboratório de ataques.

15.3 Banco Indisponível

Para alerta de banco indisponível, deve-se verificar o container do PostgreSQL, consultar os logs do banco e confirmar se o Postgres Exporter está respondendo.

15.4 Falhas de Autenticação

Quando houver muitas falhas de autenticação, deve-se verificar se a simulação de brute force foi executada. Em um ambiente real, a resposta envolveria bloqueio temporário, análise de origem e reforço de autenticação.

15.5 Vulnerabilidade Conhecida

Quando a vulnerabilidade simulada estiver ativa, o alerta crítico demonstra a necessidade de identificar o serviço afetado, corrigir a versão vulnerável e validar a resolução.

16 Resultados

O projeto resultou em uma plataforma funcional de monitoramento local. A solução permite observar métricas do web server, banco de dados, DNS, SMTP e eventos de segurança. Também permite gerar cenários de teste diretamente pelo navegador, facilitando a demonstração dos alertas.

Os principais resultados alcançados foram:

- Dashboard próprio responsivo.
- Laboratório de ataques controlados.
- Dashboard Grafana provisionado automaticamente.
- Coleta de métricas pelo Prometheus.
- Monitoramento de banco PostgreSQL.
- Probes HTTP, DNS e SMTP.
- Envio de alertas por e-mail local.
- Documentação técnica e operacional.

17 Limitações

Algumas métricas avançadas foram simplificadas para adequação ao ambiente acadêmico. Por exemplo, slow queries reais, fila SMTP real e autenticação com usuários reais não foram implementadas de forma produtiva. No entanto, o projeto apresenta simulações controladas suficientes para demonstrar o comportamento da plataforma e o acionamento dos alertas.

18 Trabalhos Futuros

Como trabalhos futuros, seria possível:

- Implementar autenticação real no front-end.
- Persistir incidentes automaticamente no PostgreSQL.
- Coletar logs estruturados.
- Adicionar métricas reais de slow queries.
- Configurar envio de e-mail por provedor real.
- Implantar a plataforma em ambiente de nuvem.
- Adicionar autenticação e controle de acesso aos dashboards.

19 Conclusão

A plataforma desenvolvida demonstra como ferramentas de observabilidade podem ser integradas para monitorar serviços de rede e aplicações. Com Prometheus, Grafana, Alertmanager, MailHog, PostgreSQL e exporters, foi possível construir um ambiente completo para coleta de métricas, visualização, alertas e testes controlados.

O projeto atende aos principais requisitos propostos no trabalho final, incluindo front-end responsivo, dashboards por serviço, backend de métricas, banco de dados, alertas por e-mail, documentação e monitoramentos extras de segurança. A solução também apresenta um laboratório de ataques controlados, que facilita a validação dos alertas e a demonstração prática da plataforma.

Referências

- [1] PROMETHEUS. *Prometheus Documentation*. Disponível em: <https://prometheus.io/docs/>.
- [2] GRAFANA LABS. *Grafana Documentation*. Disponível em: <https://grafana.com/docs/>.
- [3] PROMETHEUS. *Alertmanager Documentation*. Disponível em: <https://prometheus.io/docs/alerting/latest/alertmanager/>.
- [4] POSTGRES SQL. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>.
- [5] PROMETHEUS COMMUNITY. *Postgres Exporter*. Disponível em: https://github.com/prometheus-community/postgres_exporter.

- [6] PROMETHEUS. *Blackbox Exporter*. Disponível em: https://github.com/prometheus/blackbox_exporter.
- [7] MAILHOG. *MailHog Documentation*. Disponível em: <https://github.com/mailhog/MailHog>.
- [8] DOCKER. *Docker Compose Documentation*. Disponível em: <https://docs.docker.com/compose/>.